



Metaheuristic optimization

بهینه سازی فرا ابتکاری

Mohammad Khalooei

Spring 2026 — Amirkabir University of Technology





- **Mohammad Khalooei**

- PhD in Artificial Intelligence
- Lecturer at Sharif & Amirkabir university of technology



Khalooei@aut.ac.ir

mkhalooei90@gmail.com

@mkhalooei



هدف کلی

هدف این درس آشنایی و تولید الگوریتم‌های فراابتکاری برای حل مسایل مختلف بهینه‌سازی است. در این درس ابتدا الگوریتم‌های تکاملی معرفی می‌شوند و سپس تعدادی از الگوریتم‌های فراابتکاری شناخته شده معرفی می‌شوند و در نهایت ابزارهای لازم برای تولید انواع الگوریتم‌های فراابتکاری بررسی می‌شوند.

سرفصل‌ها

- تئوری ماکروسکوپی و میکروسکوپی تکامل
- الگوریتم‌های تکاملی: چارچوب کلی، عملگرهای تنوع، عملگرهای انتخاب
- الگوریتم‌های تکاملی: الگوریتم ژنتیکی، استراتژی تکاملی، برنامه‌نویسی ژنتیکی، الگوریتم ممتیک
- الگوریتم‌های جستجوی محلی: الگوریتم تپه‌نوردی، قدم زدن تصادفی، گرم و سرد کردن تدریجی
- الگوریتم‌های مبتنی بر جمعیت: گروه ذرات، کلونی مورچگان
- مدیریت محدودیت‌ها در بهینه‌سازی در الگوریتم‌های فراابتکاری
- مسائل بهینه‌سازی چند قلمای در الگوریتم‌های فراابتکاری
- مسائل بهینه‌سازی چند هدفی در الگوریتم‌های فراابتکاری
- روش‌های بهینه‌سازی پارامترهای الگوریتم‌های فراابتکاری

روش ارزشیابی (پیشنهادی)

- تمرین‌ها و پروژه: ۲۰ درصد نمره
- آزمون‌های میان‌ترم و پایانی: ۶۰ درصد نمره
- پروژه پژوهشی: ۲۰ درصد نمره

منابع علمی پیشنهادی

- [1] A. E. Eiben and J. E. Smith. *Introduction to Evolutionary Computing*. Springer, 2015.
- [2] K. A. D. Jong. *Evolutionary Computation: A Unified Approach*. MIT press, 2006.



عنوان درس به فارسی: بهینه‌سازی فرا ابتکاری		
عنوان درس به انگلیسی: Meta-Heuristic Optimization	نوع درس و واحد	
دروس پیش‌نیاز:	— پایه ☐ نظری ☒	
دروس هم‌نیاز:	— تخصصی الزامی ☒ عملی ☐	
تعداد واحد:	۳	تخصصی اختیاری ☐ نظری-عملی ☐
تعداد ساعت:	۴۸	مهارتی ☐

- **Lesson Objective:**

- This lesson aims to introduce various metaheuristic algorithms for solving different optimization problems. At the end of the lesson, students will become familiar with several well-known metaheuristic algorithms and will learn about the types of metaheuristics required for different optimization problems.

Main Topics:

- Introduction to Macro and Micro Evolutionary Computation
- Global, local, and population-based metaheuristics
- Single-objective and multi-objective metaheuristics
- Trajectory-based metaheuristics (such as Simulated Annealing, Tabu Search, etc.)
- Population-based metaheuristics (such as Genetic Algorithms, Particle Swarm Optimization, etc.)
- Constraint handling techniques in metaheuristic algorithms
- Several issues in improving the performance of metaheuristics
- Several target-oriented improvement issues in metaheuristics
- Methods for parameter tuning and adaptation in metaheuristic algorithms



Roadmap

Metaheuristic Optimization





- **Phase 0: Preliminaries of Optimizations**

- What is model?
- What are parameters of model?
- Parameter space vs Sample space
- What is loss function? What are they?
- What are Training of machine learning models?

- **Phase 1: Foundations of Optimization**

- Introduction to Optimization and Metaheuristics
- Modeling Optimization Problems
- Basic Concepts in Search

- **Phase 2: Single-Solution Metaheuristics**

- Random Search and Hill Climbing
- Simulated Annealing
- Tabu Search

- **Phase 3: Evolutionary and Population-Based Algorithms**

- Introduction to Evolutionary Computation
- Genetic Algorithm
- Evolution Strategies and Differential Evolution

- **Phase 4: Swarm Intelligence Algorithms**

- Particle Swarm Optimization
- Ant Colony Optimization
- Other Nature-Inspired Metaheuristics

- **Phase 5: Analysis, Comparison, and Advanced Topics**

- Performance Evaluation of Metaheuristics
- Parameter Tuning and Experimental Design
- Hybrid Metaheuristics and Final Project Presentations



- **Phase 0: Preliminaries of Optimizations**

- What is model?
- What are parameters of model?
- Parameter space vs Sample space
- What is loss function? What are they?
- What are Training of machine learning models?



- **Phase 1: Foundations of Optimization**

- Introduction to Optimization and Metaheuristics
- Modeling Optimization Problems
- Basic Concepts in Search



- **Phase 2: Single-Solution Metaheuristics**

- Random Search and Hill Climbing
- Simulated Annealing
- Tabu Search



- **Phase 3: Evolutionary and Population-Based Algorithms**

- Introduction to Evolutionary Computation
- Genetic Algorithm
- Evolution Strategies and Differential Evolution

- **Phase 4: Swarm Intelligence Algorithms**

- Particle Swarm Optimization
- Ant Colony Optimization
- Other Nature-Inspired Metaheuristics

- **Phase 5: Analysis, Comparison, and Advanced Topics**

- Performance Evaluation of Metaheuristics
- Parameter Tuning and Experimental Design
- Hybrid Metaheuristics and Final Project Presentations



- **Phase 0: Preliminaries of Optimizations**

- What is model?
- What are parameters of model?
- Parameter space vs Sample space
- What is loss function? What are they?
- What are Training of machine learning models?



- **Phase 1: Foundations of Optimization**

- Introduction to Optimization and Metaheuristics
- Modeling Optimization Problems
- Basic Concepts in Search



- **Phase 2: Single-Solution Metaheuristics**

- Random Search and Hill Climbing
- Simulated Annealing
- Tabu Search



- **Phase 3: Evolutionary and Population-Based Algorithms**

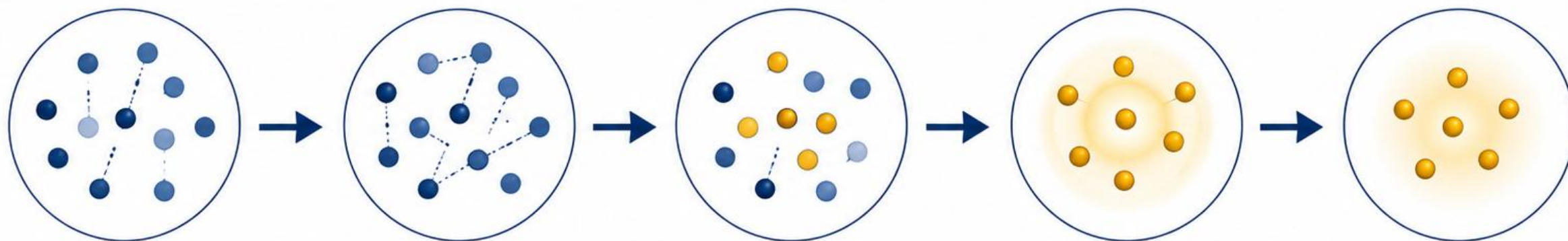
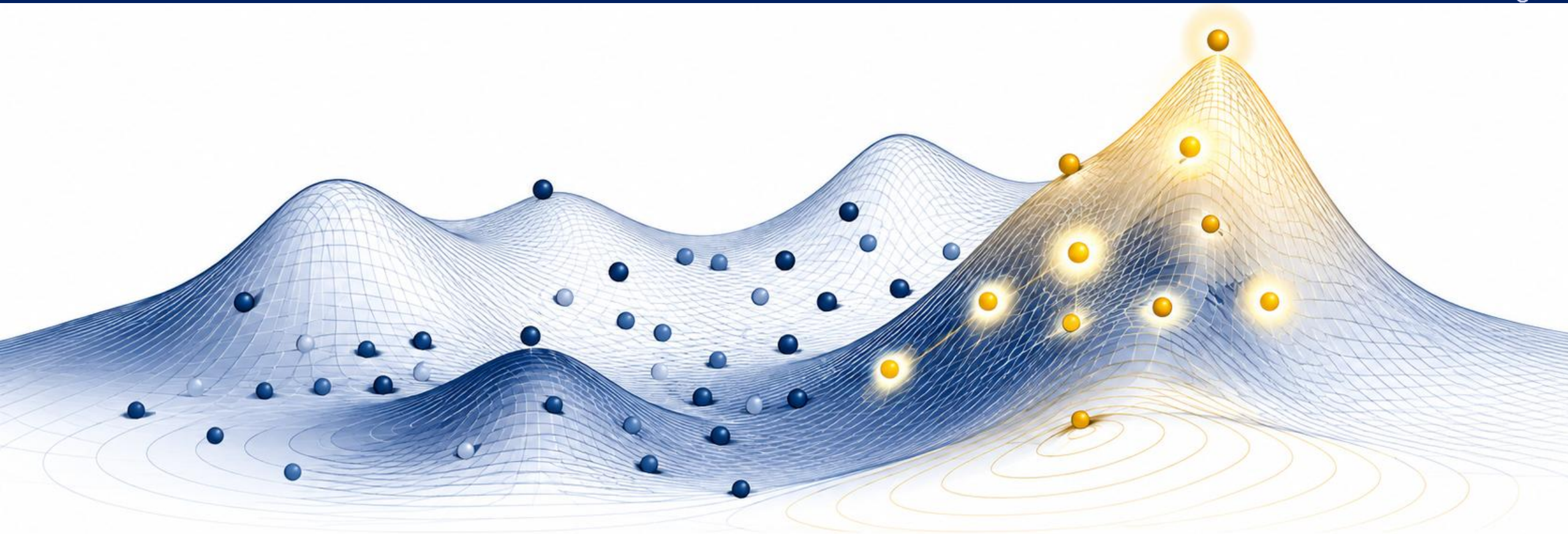
- Introduction to Evolutionary Computation
- Genetic Algorithm
- Evolution Strategies and Differential Evolution

- **Phase 4: Swarm Intelligence Algorithms**

- Particle Swarm Optimization
- Ant Colony Optimization
- Other Nature-Inspired Metaheuristics

- **Phase 5: Analysis, Comparison, and Advanced Topics**

- Performance Evaluation of Metaheuristics
- Parameter Tuning and Experimental Design
- Hybrid Metaheuristics and Final Project Presentations





The pipeline

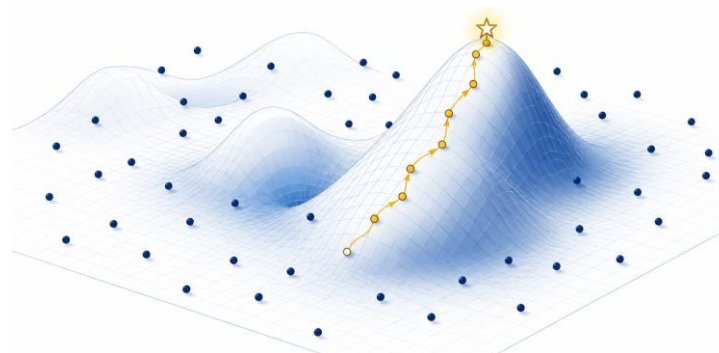
Model the problem
Generate candidates
Evaluate quality
Improve or try again



Two first algorithms

Random Search: sample candidates without memory

Hill Climbing: move step by step to better neighbors



Main question: how do we improve solutions without getting trapped?



Generate → Evaluate → Keep best

Core idea

Try many candidates randomly, evaluate each one, and remember the best solution found.

Strength

Broad exploration
Easy baseline
Parallelizable

Limitation

No direction
No exploitation
Wastes evaluations



Random Search is not “bad”; it tells us what unguided search can achieve.



Start from one solution, move to a better neighbor.

Neighborhood

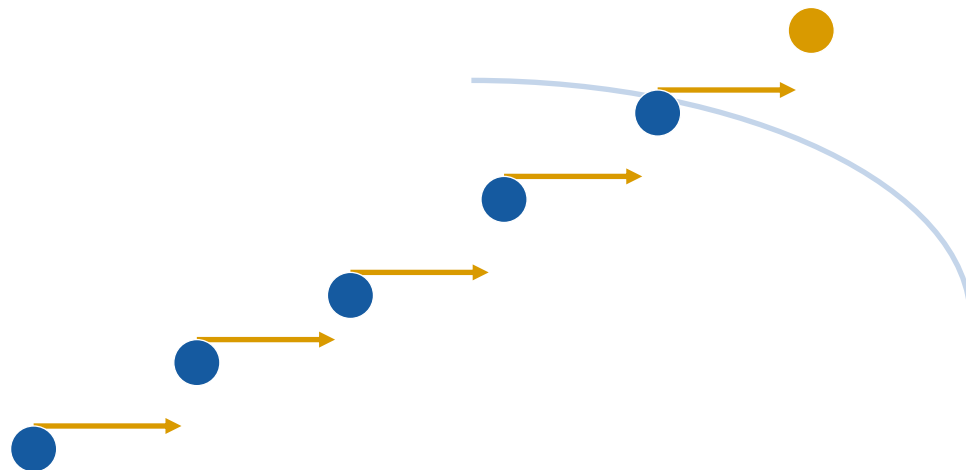
Defines what “nearby” means: swap two cities, flip one item, or slightly change a parameter.

Greedy acceptance

Accept the neighbor only if it improves the objective value.

Local optimum risk

If every neighbor is worse, the algorithm stops—even if better solutions exist elsewhere.



local path



Random Search vs. Hill Climbing



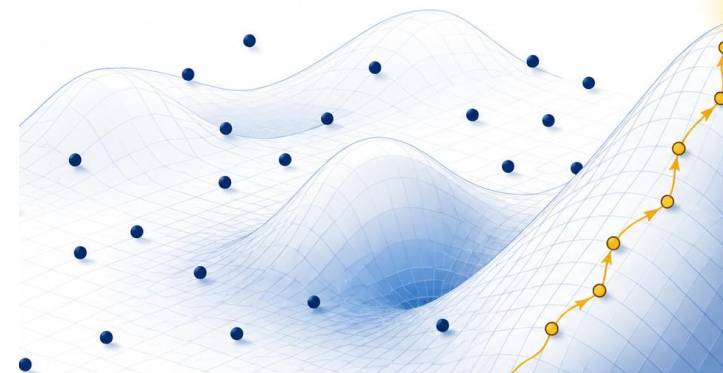
Both are simple, but their search behavior is very different.

Aspect	Random Search	Hill Climbing
Starting point	Many independent candidates	One current solution
Guidance	No local guidance	Uses neighborhood improvement
Memory	Best-so-far only	Current + best-so-far
Strength	Broad sampling	Fast local improvement
Weakness	Wastes evaluations	Can get stuck locally
Best role	Exploration / baseline	Exploitation / local search

Random Search explores broadly; Hill Climbing exploits locally.



Why Move to Population-Based Search?





Why Move to Population-Based Search?



- One candidate gives one viewpoint.
- A population gives many viewpoints.

Single-solution methods

Current solution
Local moves
Fast refinement
Risk: local trap



Population-based methods

Multiple candidates
Diversity + selection
Parallel exploration
Better escape potential



one search point



Phase 3: make many solutions compete, recombine, and adapt.



- Terminologies

Population

A set of candidate solutions exists at the same time.

Fitness

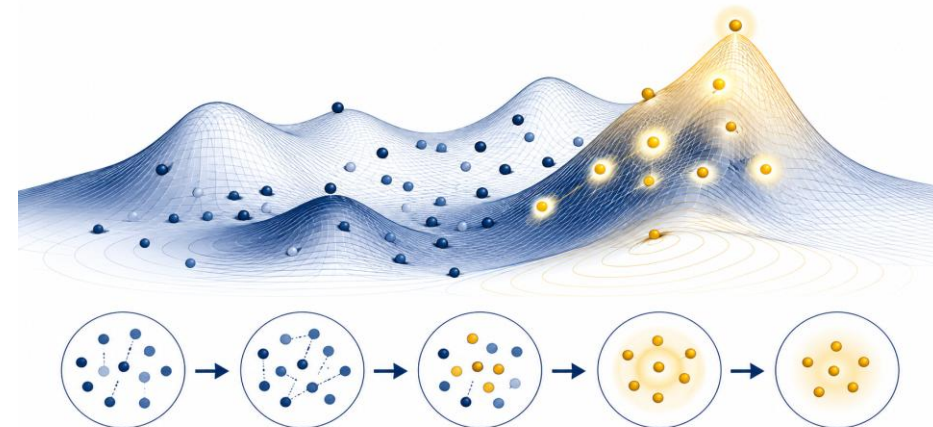
Each candidate is evaluated by the objective function.

Selection

Better candidates get more chance to influence the next generation.

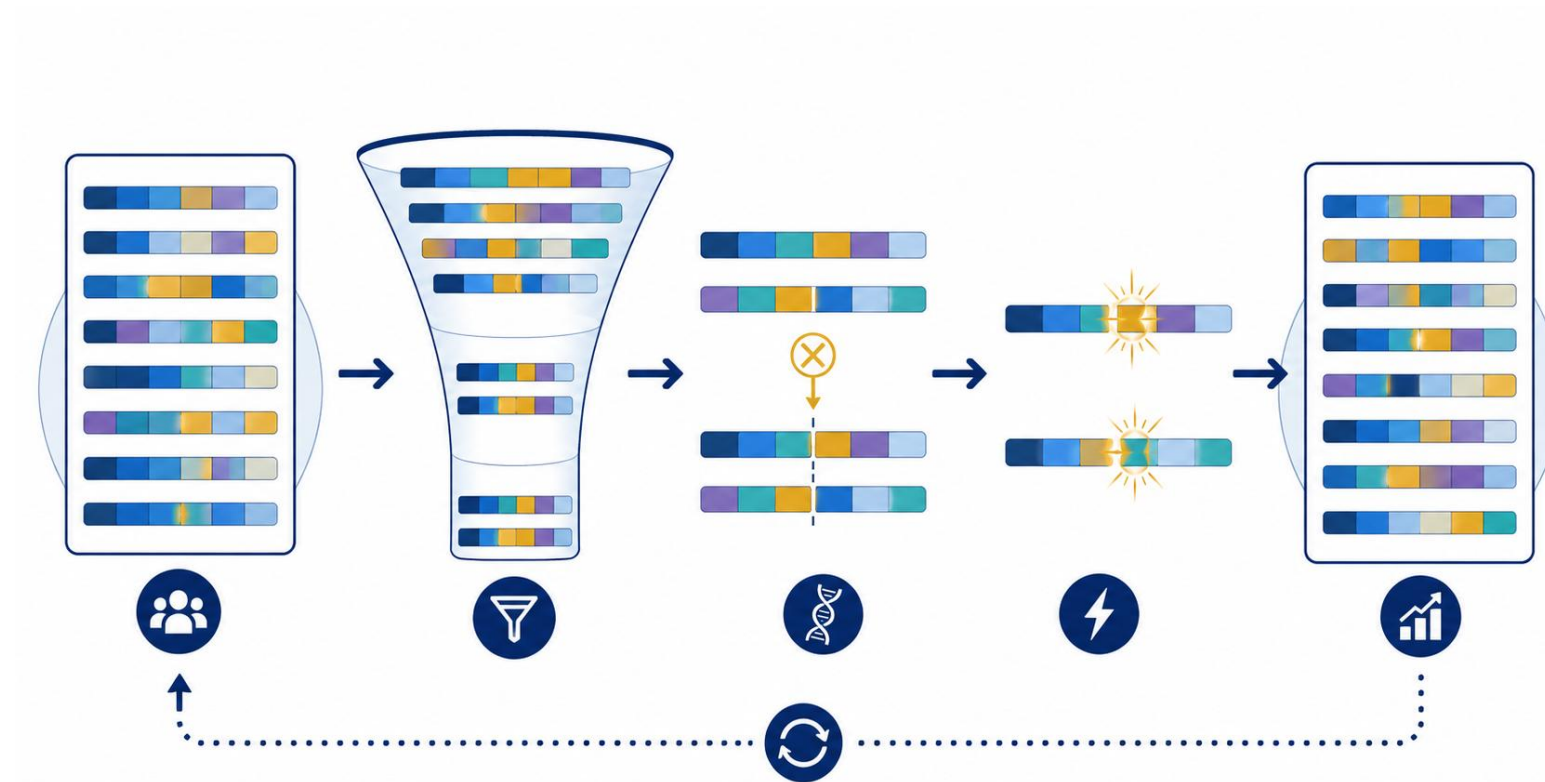
Variation

Crossover and mutation create new candidates.





Genetic Algorithm: The Main Loop



Population

Selection

Crossover

Mutation

Next generation

- Before a GA can search, we must decide how a solution is encoded.

Binary encoding

Knapsack: 1 means selected, 0 means not selected.

1	0	1	1	0	1
---	---	---	---	---	---

- Before a GA can search, we must decide how a solution is encoded.

Binary encoding

Knapsack: 1 means selected, 0 means not selected.

1	0	1	1	0	1
---	---	---	---	---	---

Permutation encoding

TSP: an ordering of cities defines a tour.

A	D	C	B	E	F
---	---	---	---	---	---



- Before a GA can search, we must decide how a solution is encoded.

Binary encoding

Knapsack: 1 means selected, 0 means not selected.

1	0	1	1	0	1
---	---	---	---	---	---

Permutation encoding

TSP: an ordering of cities defines a tour.

A	D	C	B	E	F
---	---	---	---	---	---

Real-valued encoding

Model parameters or engineering variables are stored as vectors.

0.2	1.8	-0.7	3.1
-----	-----	------	-----

- Before a GA can search, we must decide how a solution is encoded.

Binary encoding

Knapsack: 1 means selected, 0 means not selected.

1	0	1	1	0	1
---	---	---	---	---	---

Permutation encoding

TSP: an ordering of cities defines a tour.

A	D	C	B	E	F
---	---	---	---	---	---

Real-valued encoding

Model parameters or engineering variables are stored as vectors.

0.2	1.8	-0.7	3.1
-----	-----	------	-----

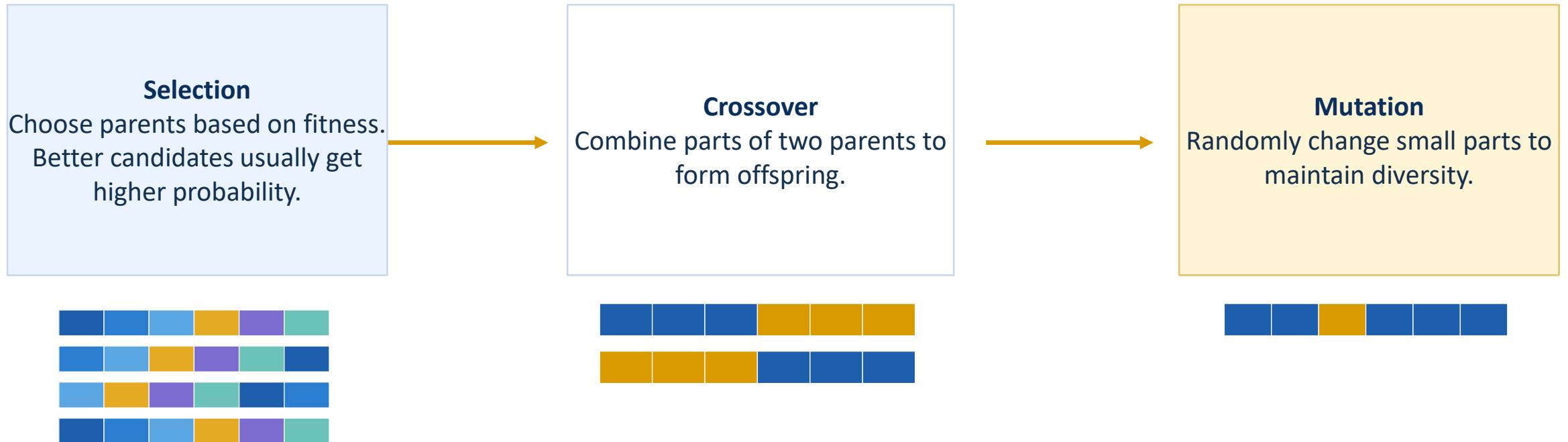
Fitness function = objective value translated into a quality score.



Selection, Crossover, and Mutation



- Variation creates new candidates; selection decides what information survives.



Selection exploits good solutions; mutation protects exploration.



Evolution Strategies: Continuous Evolution



Best suited for

Continuous optimization with real-valued vectors.

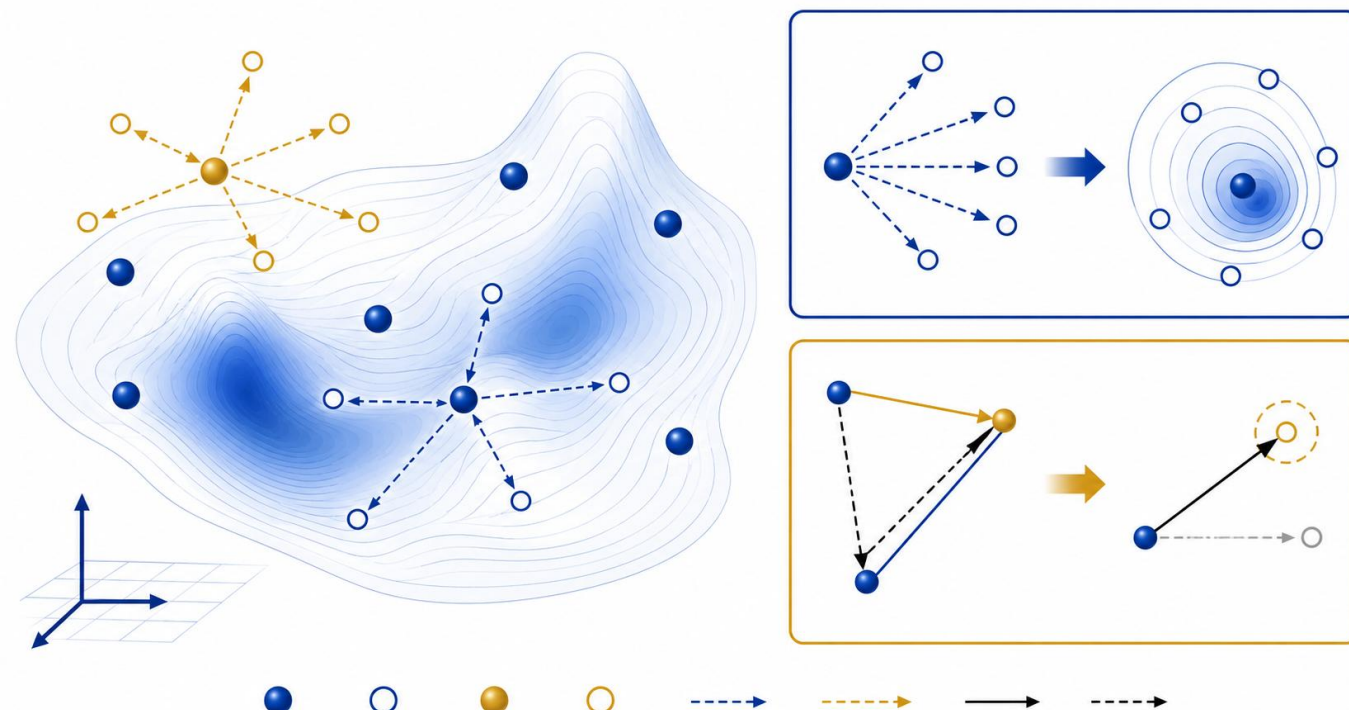
Main mechanism

Create offspring by adding random changes, often Gaussian mutation.

Key idea

Adapt not only the solution, but sometimes the mutation step size.

$$\mathbf{x}' = \mathbf{x} + \sigma \cdot \mathbf{N}(0, \mathbf{I})$$





Differential Evolution: Learning from Differences



- DE creates a new candidate using differences between population members.

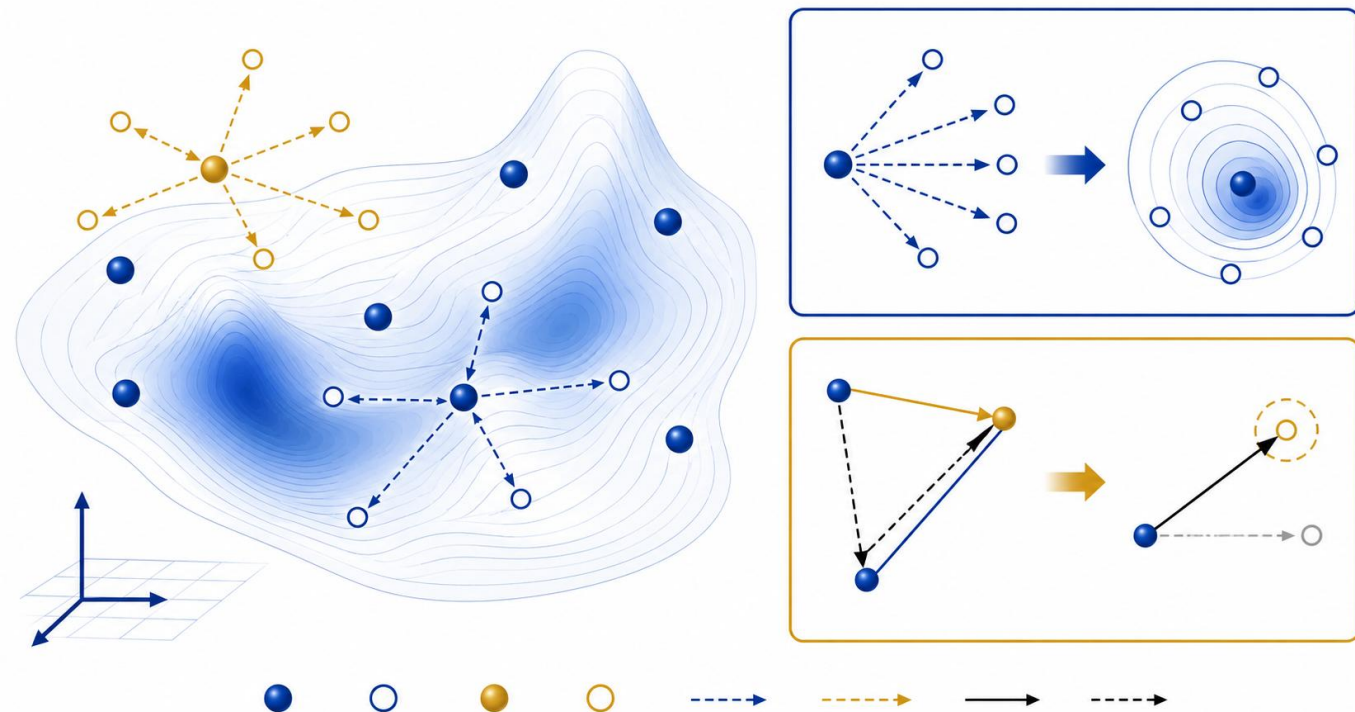
Mutation vector

Take three individuals and build a trial direction from their difference.

$$\mathbf{v} = \mathbf{a} + F \cdot (\mathbf{b} - \mathbf{c})$$

Crossover + selection

Mix the trial vector with the target vector, then keep the better one.





- Use the same small problem as Phase 2, but now search with a population.

- 1 Choose representation** Binary knapsack, TSP permutation, or numeric vector
- 2 Create population** Start with 6–8 random candidate solutions
- 3 Evaluate fitness** Compute cost, distance, error, or profit
- 4 Apply operators** Select parents, crossover, mutate, replace
- 5 Compare behavior** How is it different from Hill Climbing?

Discussion: Did the population explore more than one local path?



Key Takeaways and Next Step



- Phase 3 extends the Phase 2 search pattern from one solution to many.

Population

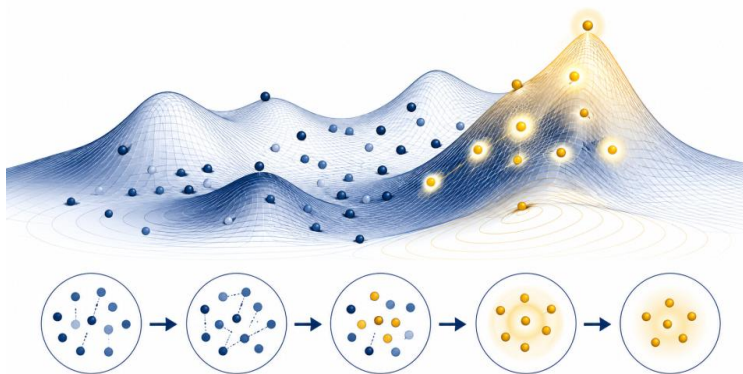
Multiple candidates search in parallel and preserve diversity.

Fitness

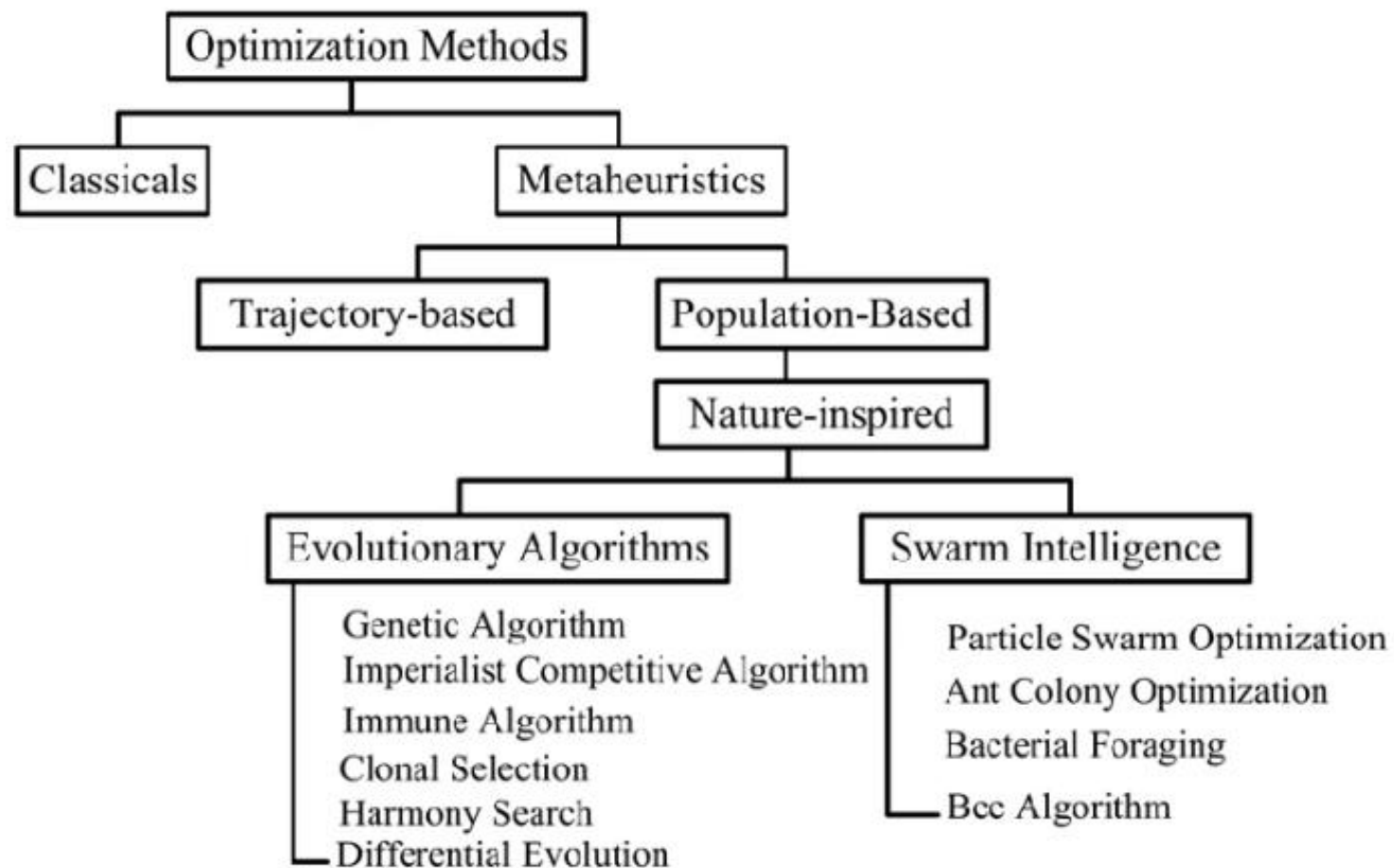
Objective quality controls selection and survival.

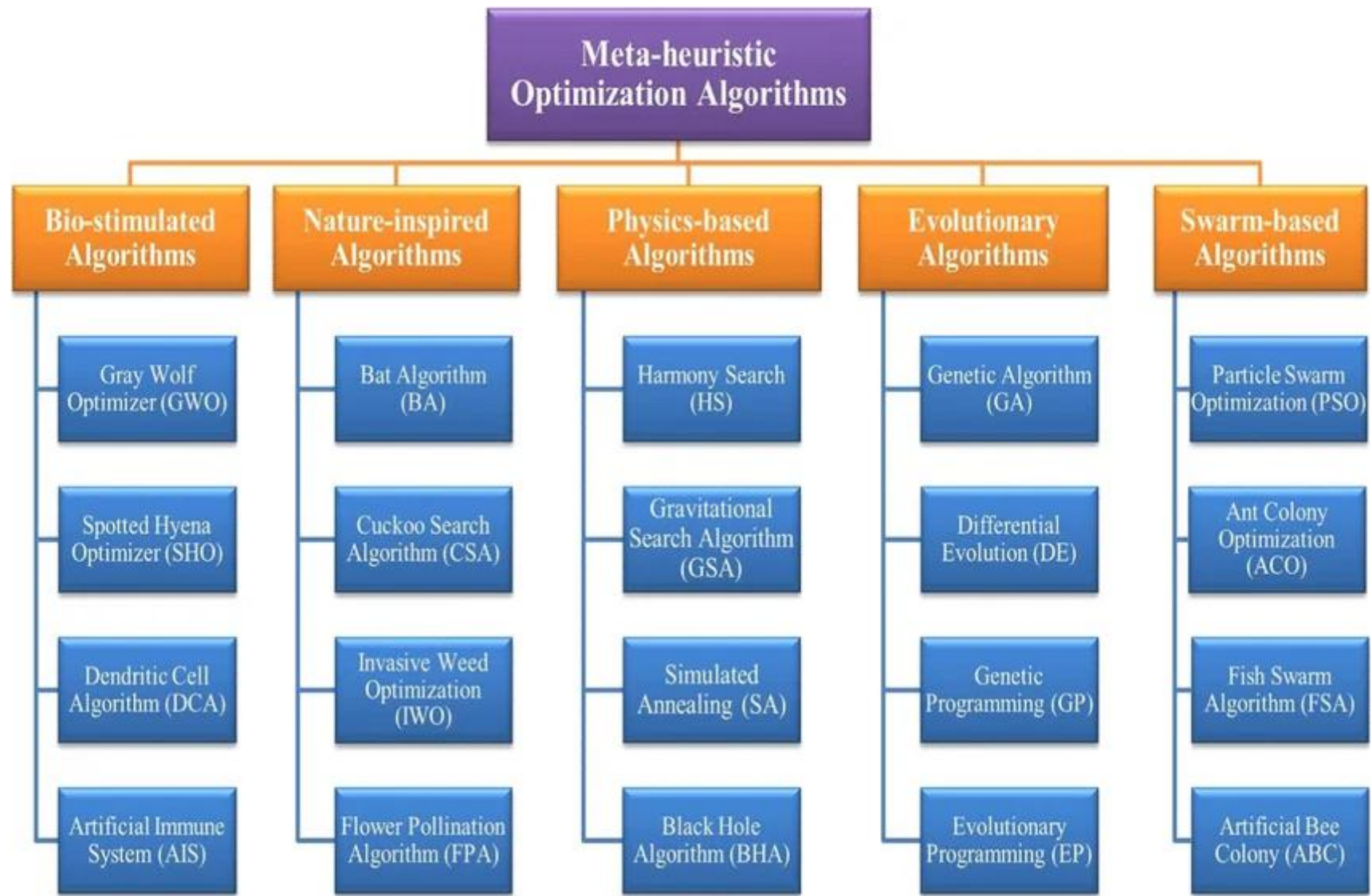
Variation

Crossover, mutation, and vector differences create new candidates.



Next session: implement the Genetic Algorithm loop step by step.





https://www.researchgate.net/publication/333940328_A_comparative_review_of_meta-heuristic_approaches_to_optimize_the_SLA_violation_costs_for_dynamic_execution_of_cloud_services?_tp=eyJjb250ZXh0Ijp7ImZpcnN0UGFnZSI6Ii9kaXJlY3QiLCJwYVWdlIjoiX2RpcmVjdCJ9fQ

